Mawlana Bhashani Science and Technology University

Department of Information and Communication Technology

Course Code: ICT-3107
Course Name: Software Engineering and Project Management

Library Management System

Final Capstone Project Proposal and Prototype (Part I)

Submitted by:

Shayla Akter
ID: IT24022
Department of ICT, MBSTU

Supervised by:

Professor Dr. Ziaur Rahman
Department of ICT
Mawlana Bhashani Science and Technology University

Date of Submission: October 28, 2026

Contents

1 Introduction **2**
- 1.1 Problem Statement . . . 2
- 1.2 Objectives . . . 2
- 1.3 Scope . . . 2

2 Requirements **2**
- 2.1 Functional Requirements . . . 2
- 2.2 Non-Functional Requirements . . . 3
- 2.3 Feasibility Study . . . 3

3 Software Process Model **3**

4 Project Schedule, Milestones and Deliverables **4**
- 4.1 Milestones and Deliverables . . . 4
- 4.2 Gantt Chart . . . 4
- 4.3 Risks . . . 5

5 System Analysis and Design (UML) **5**
- 5.1 Data Flow Diagram . . . 5
- 5.2 Use Case Diagram . . . 6
- 5.3 Activity Diagram . . . 7
- 5.4 Entity Relationship Diagram . . . 9
- 5.5 Class Diagram . . . 9
- 5.6 Object Diagram . . . 11

6 Prototype Implementation **11**
- 6.1 Screenshots . . . 12
- 6.2 Test Cases . . . 12

7 Conclusion and Future Work **12**

1 Introduction

Libraries still keep many records on paper or in spreadsheets. This makes searching for books slow, makes it hard to know who holds which book, and leads to lost records and uncollected fines. The **Library Management System (LMS)** is a desktop application written in Java that automates the daily work of a small library: cataloguing books, registering members, issuing and returning books, calculating fines and reporting overdue items.

1.1 Problem Statement

Manual library handling causes (i) slow search of books, (ii) errors in issue and return records, (iii) difficulty in tracking overdue books and calculating fines, and (iv) no quick reports for the administrator. A simple, reliable, computerized system is needed.

1.2 Objectives

- Store and manage book, member and issue data in one place.
- Let librarians issue and return books in a few clicks.
- Calculate late-return fines automatically.
- Provide fast search by title, author or category.
- Provide role-based login for Admin and Librarian.
- Generate an overdue-books report.

1.3 Scope

In scope: login with roles, book management, member registration, issue/return, fine calculation, search, overdue report, file-based data storage.
Out of scope (future work): online member portal, barcode/RFID scanning, e-mail/SMS reminders, online payment of fines.

2 Requirements

2.1 Functional Requirements

ID	Requirement
FR1	The system shall allow Admin and Librarian to log in with a username and password.
FR2	The system shall allow adding books and (Admin only) deleting books.
FR3	The system shall search books by title, author or category.
FR4	The system shall register new members.
FR5	The system shall issue a book to a member if a copy is available and the member has fewer than 3 active loans.
FR6	The system shall set the due date 14 days after the issue date.
FR7	The system shall record a return and calculate a fine of Tk 5 per late day.
FR8	The system shall list overdue books with the fine accumulated so far.

ID	Requirement
FR9	The system shall save all data permanently and reload it at start-up.

2.2 Non-Functional Requirements

Usability: simple tabbed interface. **Reliability:** data saved after every change. **Portability:** runs on any system with Java 17 or later. **Maintainability:** separate classes for data, logic and interface. **Performance:** search and issue operations complete in under one second for up to 10,000 records.

2.3 Feasibility Study

Technical: Java and Swing are free and well known. **Economic:** no cost, only open-source tools. **Operational:** the interface is simple enough for library staff with minimal training. **Schedule:** the 12-week plan in Section 4 is realistic for one student.

3 Software Process Model

We use the **Agile (Scrum-based incremental) model**. The project is divided into four sprints of about two to three weeks. Each sprint ends with a working increment that is reviewed and improved. This suits a single-student project because requirements can be refined as the prototype grows, and risk is reduced by testing early.

Sprint	Focus	Increment delivered
Sprint 0	Planning and analysis	Proposal, requirements, UML diagrams
Sprint 1	Core data and login	Classes, storage, login screen
Sprint 2	Books and members	Add/search/delete books, register members
Sprint 3	Issue/return and fines	Issue, return, fine calculation, overdue report
Sprint 4	Testing and delivery	Bug fixes, final report, GitHub release

Table 2: Sprint plan

4 Project Schedule, Milestones and Deliverables

4.1 Milestones and Deliverables

Milestone	Week	Deliverable
M1 Requirements approved	2	Proposal and requirement list
M2 Design completed	3	DFD, use case, activity, ERD, class and object diagrams
M3 Prototype v0.1	5	Login screen and data storage working
M4 Prototype v0.5	8	Book and member modules working
M5 Prototype v1.0	10	Issue, return, fine and report modules
M6 Final release	12	Tested system, final report, GitHub repository

Table 3: Milestones

4.2 Gantt Chart

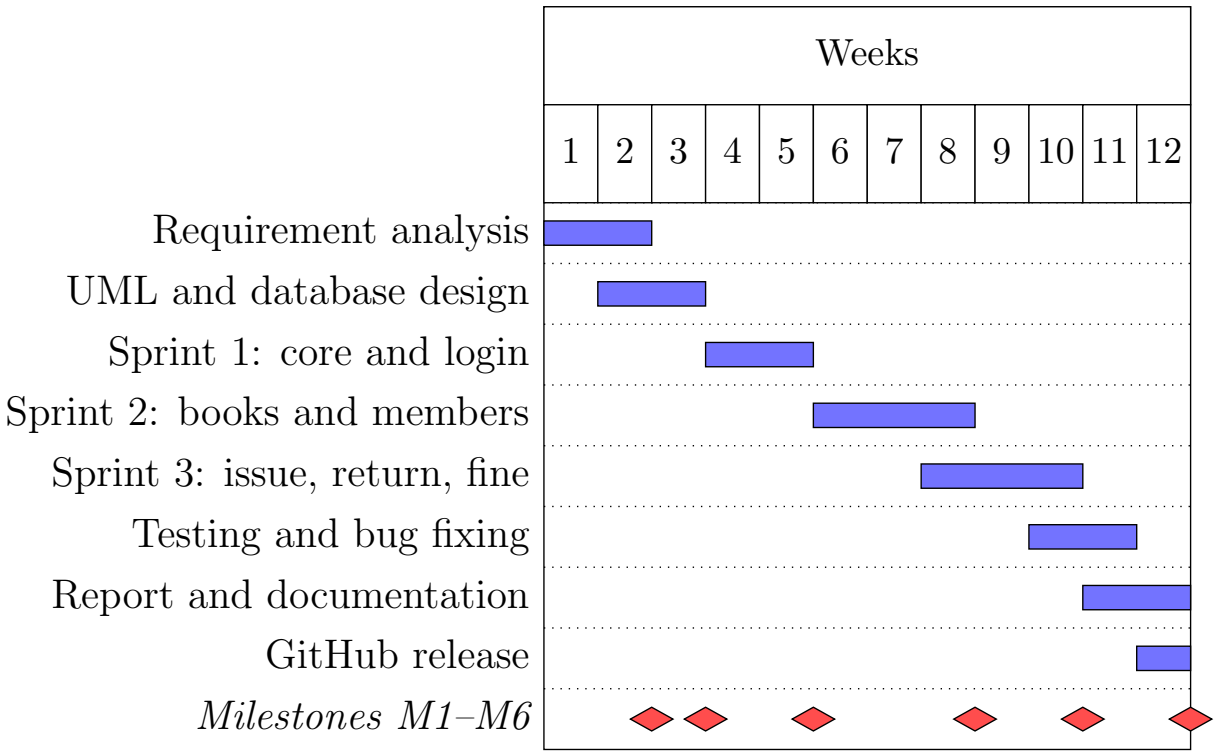


Figure 1: Project Gantt chart (12 weeks)

4.3 Risks

Risk	Mitigation
Data file damaged or lost	Save after every change; keep backups in GitHub
Time shortage	Build the core features first; extras last
Bugs in fine or date logic	Test with different issue/return dates
Interface not user friendly	Keep layout simple; collect feedback each sprint

Table 4: Risk plan

5 System Analysis and Design (UML)

5.1 Data Flow Diagram

Figure 5.1 shows the context diagram (Level 0) and Figure 5.1 the Level 1 DFD.

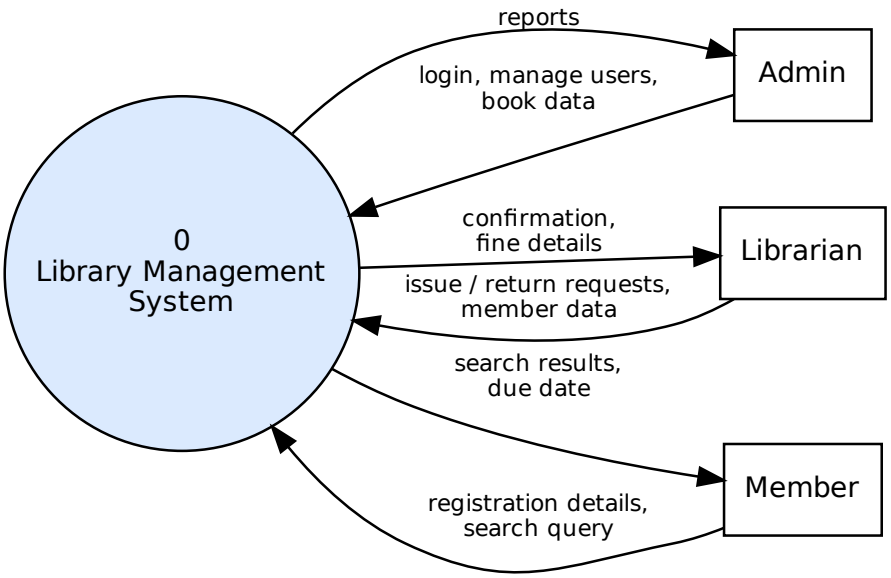


Figure 2: DFD Level 0 (context diagram)

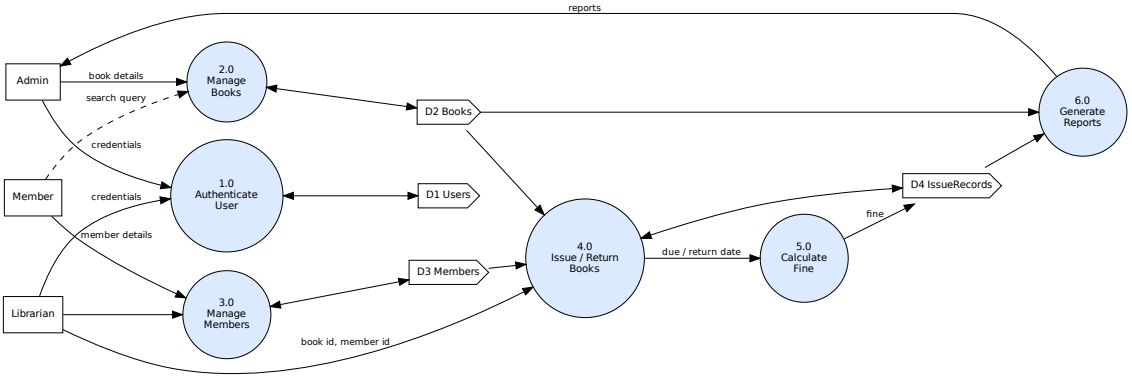


Figure 3: DFD Level 1

5.2 Use Case Diagram

Actors are Admin, Librarian and Member. Returning a book includes fine calculation.

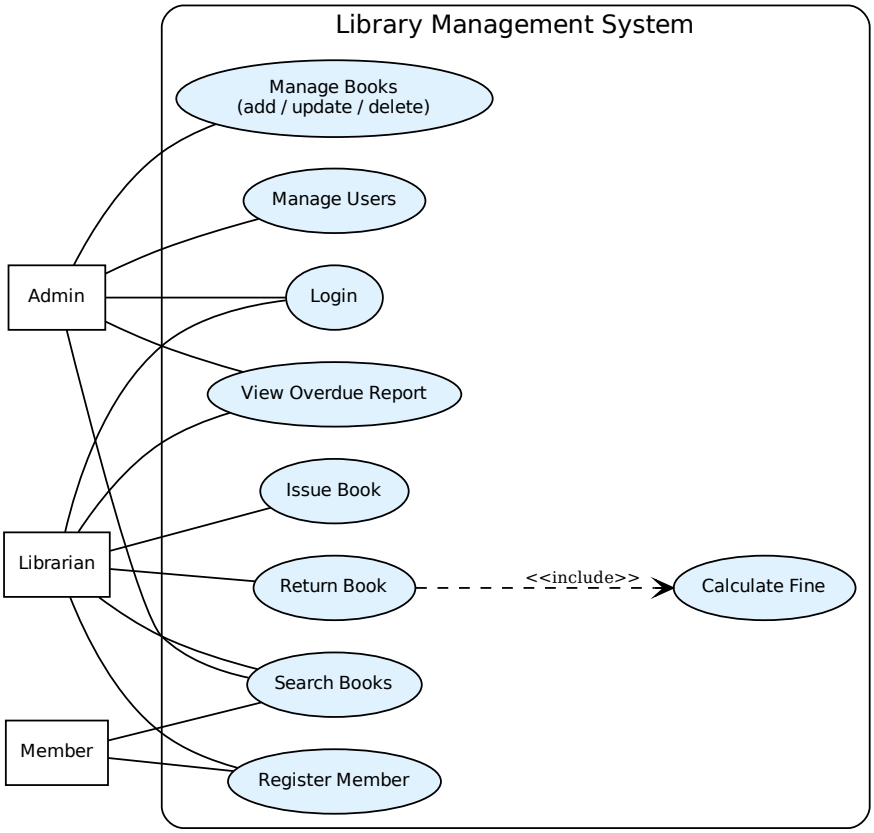
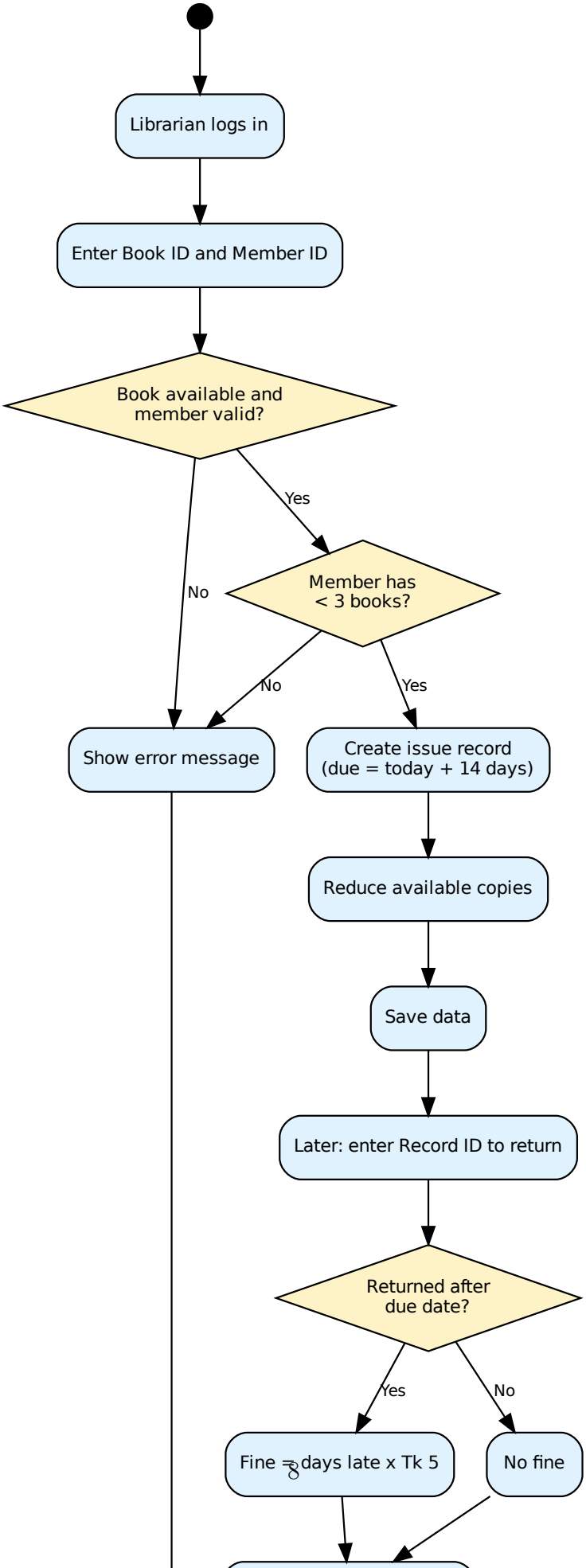


Figure 4: Use case diagram

5.3 Activity Diagram

The activity diagram shows the issue and return workflow.


Librarian logs in
Enter Book ID and Member ID
Book available and member valid?
Yes
No
Member has < 3 books?
No
Yes
Show error message
Create issue record (due = today + 14 days)
Reduce available copies
Save data
Later: enter Record ID to return
Returned after due date?
Yes
No
Fine = days late x Tk 5
No fine

5.4 Entity Relationship Diagram

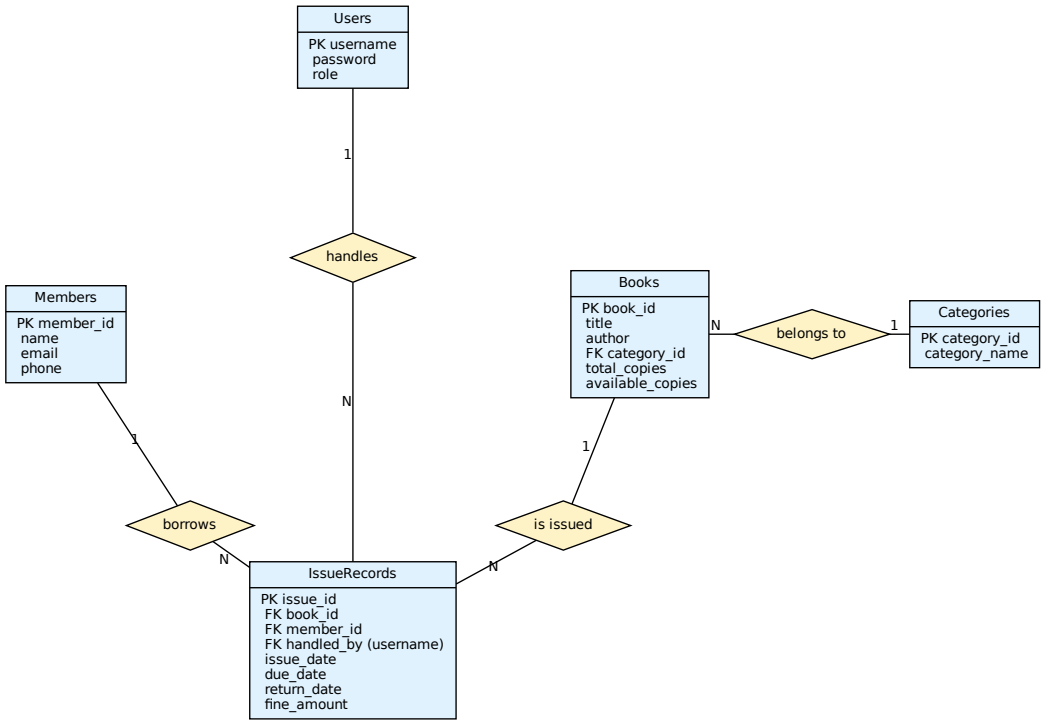


Figure 6: Entity relationship diagram

Relationship	Cardinality
Category – Book	One category has many books
Member – IssueRecord	One member borrows many times
Book – IssueRecord	One book is issued many times
User – IssueRecord	One staff user handles many issues

Table 5: ERD relationships

5.5 Class Diagram

The prototype implements all of these classes: `User`, `Book`, `Member`, `IssueRecord`, `Fine`, `LibraryService`, `LoginFrame`, `MainFrame` and `Main`.

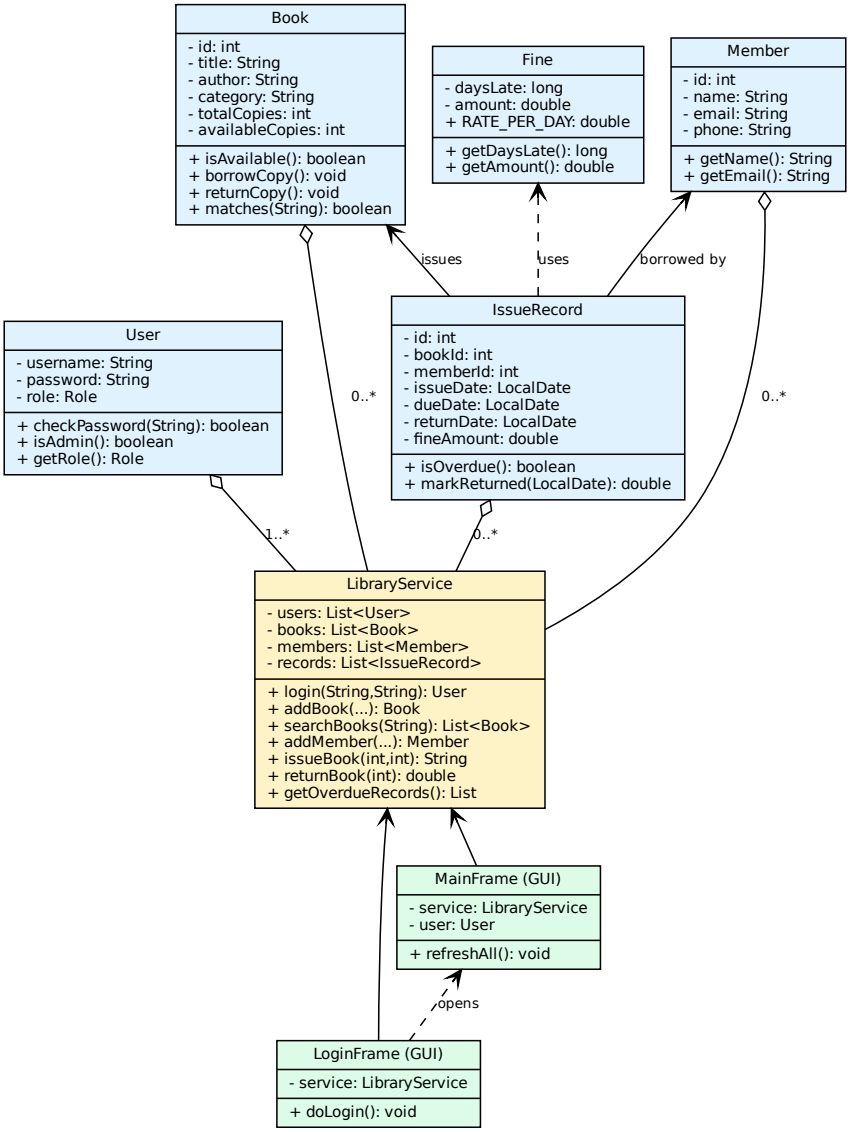


Figure 7: Class diagram

5.6 Object Diagram

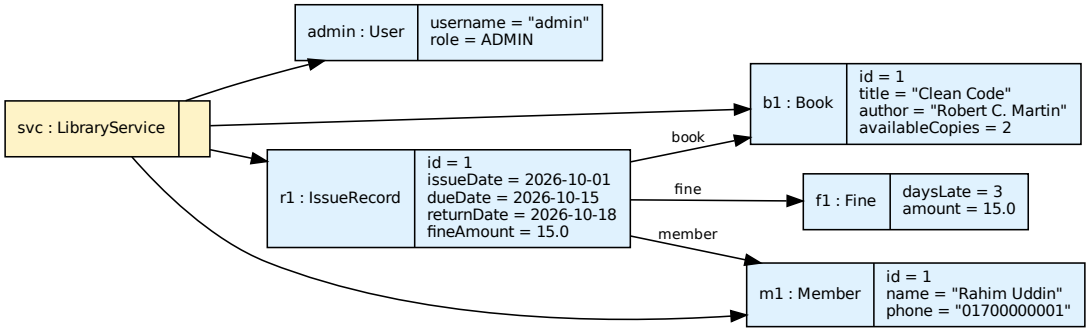


Figure 8: Object diagram: a late return with a 3-day fine

6 Prototype Implementation

The prototype is a working Java Swing application with the following structure.

```
LibraryManagementSystem/
  src/library/
    Main.java              (entry point)
    LoginFrame.java        (login screen)
    MainFrame.java         (tabs: Books, Members, Issue/Return,
       Overdue)
    LibraryService.java    (business logic + CSV storage)
    User.java  Book.java  Member.java  IssueRecord.java  Fine.
       java
  data/                    (CSV files created automatically)
  docs/diagrams/           (Graphviz sources of all diagrams)
  report/                  (this LaTeX proposal)
```

Technology: Java 17+, Swing, CSV file storage, IntelliJ IDEA, Git/GitHub, LaTeX.
Classes used from the class diagram (9 classes, minimum required is 5): `User`, `Book`, `Member`, `IssueRecord`, `Fine`, `LibraryService`, `LoginFrame`, `MainFrame`, `Main`.

Features in the prototype: login for Admin and Librarian (demo accounts: `admin` / `admin123` and `librarian` / `lib123`); add, search and delete books; register members; issue book with a 14-day due date and a 3-book limit; return book with automatic fine (Tk 5 per day); overdue report; automatic saving to files.

6.1 Screenshots

6.2 Test Cases

No.	Test	Expected result	Status
T1	Login with wrong password	Error message shown	Pass
T2	Issue book with no copies left	“No copies available”	Pass
T3	Issue to unknown member	“Member not found”	Pass
T4	Return book 3 days late	Fine = Tk 15	Pass
T5	Return an already returned record	Rejected	Pass
T6	Delete a book that is on loan	Not allowed	Pass
T7	Close and reopen program	All data reloaded	Pass

Table 6: Test cases

7 Conclusion and Future Work

The Library Management System gives a small library a simple, reliable way to manage books, members, loans and fines. The plan, diagrams and working prototype in this proposal show that the project is complete in scope and can be finished in the given time. Future work includes a database (SQLite/MySQL), a member web portal, barcode scanning and e-mail reminders.

References

[1] I. Sommerville, *Software Engineering*, 10th ed., Pearson, 2016.

[2] R. S. Pressman, *Software Engineering: A Practitioner’s Approach*, 8th ed., McGraw-Hill, 2014.

[3] M. Fowler, *UML Distilled*, 3rd ed., Addison-Wesley, 2003.

[4] Oracle, *Java Swing Tutorial*, https://docs.oracle.com/javase/tutorial/uiswing/.

[5] K. Schwaber and J. Sutherland, *The Scrum Guide*, 2020.